T3 子序列计数（count）题解

2026 赛季 CSP-S 复赛热身赛（03） · 时间 1.0 秒 / 内存 512 MiB · 答案对 998244353 取模

目 录

0 题意：这题要我们做什么
1 数据范围与解题思路
2 第一档：子集枚举（15 分）
3 第二档：按“最后三个”记 dp（35 分）
4 第三档：压掉一维，查表定撞车（60 分）
5 正解：修正项整批预处理（100 分）
6 易错点清单
7 总结

0 题意：这题要我们做什么

一句话题意

给你一个长度为 n 的序列 a（**每个数互不相同**），数一数它有多少个**非空子序列**，满足：子序列里任意**连续 4 项**的异或都不等于 s。答案对 998244353 取模。

> **子序列 ≠ 子串**。子序列只要求“下标递增地挑出来”，挑中的元素在原序列里可以不挨着；所谓“连续 4 项”，指的是**挑出来之后**在新序列里挨着的 4 项。例如从 (2,3,4,7,6) 里挑出下标 1、2、3、5，得到子序列 (2,3,4,6)，它的连续 4 项就是 2,3,4,6 这一组。

异或是什么

异或（写作 $\oplus$，C++ 里写 `^`）就是二进制下的“不进位加法”：每一位上，两个数字**不同得 1，相同得 0**。例如 $6 \oplus 3$：6 是 110，3 是 011，按位比一比得 101，也就是 5。

本题只用到一个性质：**异或是它自己的逆运算**——如果 $x \oplus y = z$，那么 $y = x \oplus z$。所以窗口异或等于 s 这件事：

$$x \oplus y \oplus z \oplus w = s \iff w = s \oplus x \oplus y \oplus z$$

也就是说，**窗口的前三项定下来后，会让窗口“撞车”的第四项是唯一确定的**。这是整道题的钥匙。

样例 1 手算

输入 $n = 5$，$m = 3$，$s = 3$，$a = (2, 3, 4, 7, 6)$。长度为 5 的序列共有 $2^5 - 1 = 31$ 个非空子序列。长度小于 4 的一定合法（题面有注），只需检查长度 $\geqslant 4$ 的 6 个：

子序列	连续 4 项的异或	合法?
(2,3,4,7)	$2⊕3⊕4⊕7 = 2$	√
(2,3,4,6)	$2⊕3⊕4⊕6 = 3 = s$	✗
(2,3,7,6)	$2⊕3⊕7⊕6 = 0$	√
(2,4,7,6)	$2⊕4⊕7⊕6 = 7$	√
(3,4,7,6)	$3⊕4⊕7⊕6 = 6$	√
(2,3,4,7,6)	$2⊕3⊕4⊕7 = 2$，$3⊕4⊕7⊕6 = 6$	√

只有 (2,3,4,6) 不合法，答案 = 31 − 1 = **30**，与输出一致。顺便记住一个边界：**$n < 4$ 时任何子序列都合法，答案恒为 $2^n - 1$**（比如 n = 3 时答案是 7），可以拿来自测程序。

1　数据范围与解题思路

各个参数的范围

量	范围
n	$1 \leqslant n \leqslant 4000$
m	$1 \leqslant m \leqslant 12$（值域只有 $0 \sim 2^m-1$，最大 4095）
a_i、s	$0 \leqslant a_i, s < 2^m$，且 a_i **互不相同**
时限 / 内存	1.0 秒 / 512 MiB，答案对 998244353 取模

子任务（subtask）

编号	分值	限制
#1	15	$n \leqslant 10$，$m \leqslant 4$
#2	20	$n \leqslant 60$，$m \leqslant 6$
#3	25	$n \leqslant 200$，$m \leqslant 8$
#4	25	$n \leqslant 1000$，$m \leqslant 10$
#5	15	$n \leqslant 4000$，$m \leqslant 12$

本题是捆绑测试：同一档内的测试点必须全对才得分。

读法　做题前先问自己三个问题：

- **枚举得起吗？** $n \leqslant 10$ 时 2^n 只有 1024，随便枚举；$n = 60$ 时 2^{60} 是天文数字，必须用 dp。
- **dp 状态记几项？** 禁用窗口有 4 项，记“结尾 3 项”最自然：$O(n^3)$ 格状态 × $O(n)$ 转移 = $O(n^4)$。时间上 $n \leqslant 200$ 都绰绰有余，但三维数组的**内存**随 n^3 膨胀，这才是它真正的天花板。
- **还能压多低？** $n = 200$ 想要 $O(n^3)$，$n = 4000$ 想要 $O(n^2)$ 级别。另外注意 $m \leqslant 12$：**值域只有 4096**，且值互不相同——可以开一张“值 → 位置”的表 pos，它是后面每一步优化的关键道具。

升级路线，本文也按它组织：

档位	做法	解决的问题 / 还差什么
第 2 节	二进制子集枚举，逐个检查	会数了，但 $O(2^n \cdot n)$ 太慢 → 15 分
第 3 节	三维 dp：记住结尾三个位置	会 dp 了，但三维数组内存随 n^3 爆炸 → 35 分
第 4 节	压成二维 dp，pos 表查唯一撞车值	$O(n^3)$，n 到 4000 要半分钟 → 60 分
第 5 节	同一个 dp，修正项用 G 表整批预处理	$O(n^2 + n \cdot 2^m)$，全部数据 → 100 分

2　第一档：子集枚举

15 分

适用 subtask #1（15 分）：$n \leqslant 10$，一共 2^{10} 种选法，去掉空集是 1023 个非空子序列。

n 个元素，每个只有“选 / 不选”两种状态——这正好对应一个 **n 位二进制数**：第 i 位是 1 就表示选 a[i]。让 mask 从 1 走到 $2^n - 1$，就把所有非空子序列不重不漏地列出来了，这就是**子集枚举**。判断“第 i 位是不是 1”用位运算：`mask >> (i-1) & 1`。每得到一个子序列，扫一遍检查每个长度 4 的连续窗口即可。

partial1_subset.cpp · 二进制子集枚举，$O(2^n \cdot n)$

```cpp
// T3 第一档：二进制子集枚举，O(2^n * n)
// 适用：Subtask #1，n <= 10，2^10 = 1024 种子集随便跑
#include <cstdio>
const int MAXN = 25;
int n, m, s;
int a[MAXN];
int sub[MAXN];        // 当前子集选出的子序列

int main() {
    freopen("count.in", "r", stdin);
    freopen("count.out", "w", stdout);
    scanf("%d %d %d", &n, &m, &s);
    for (int i = 1; i <= n; i++) scanf("%d", &a[i]);

    long long ans = 0;
    for (int mask = 1; mask < (1 << n); mask++) {
        int len = 0;
        for (int i = 1; i <= n; i++)          // mask 第 i 位是 1 就选 a[i]
            if (mask >> (i - 1) & 1) sub[++len] = a[i];
        bool ok = true;
        for (int j = 1; j + 3 <= len; j++)    // 检查每个连续四项
            if ((sub[j] ^ sub[j+1] ^ sub[j+2] ^ sub[j+3]) == s) {
```

```
                ok = false;                    // 出现禁用窗口
                break;
            }
        if (ok) ans++;
    }
    printf("%lld\n", ans % 998244353);
    return 0;
}
```

不够　n = 60 时有 $2^{60} \approx 10^{18}$ 种子序列，枚举法彻底没戏。子序列个数是指数级的，必须换个角度：**不枚举，改用 dp 计数**。

3　第二档：按“最后三个”记 dp　35 分

适用 subtask #1、#2（合计 35 分）：n ⩽ 60。

状态：盯住结尾的三个位置

子序列计数有个经典套路：按“结尾”来数，对每个结尾位置算出“以它结尾的有几个”，全部加起来就是答案。本题的禁用窗口有 4 项——接一个数到前缀后面，合不合法要看前缀的**倒数第 1、2、3 项**，所以状态记住结尾三个位置：

定义　g[u][t][i]（u < t < i）= 最后三个被选位置**恰好**是 u、t、i 的合法子序列个数（长度 ⩾ 3）。答案 = n（长度 1）+ C(n,2)（长度 2，必合法）+ 所有 g[u][t][i] 之和。

转移：接 a[i]，逐个检查第四项

g[u][t][i] 里的子序列，都是“某个以 (u,t) 结尾的合法前缀 P”再接上 a[i]：

- **P 只有 {u, t}**：1 种，接上后长度 3，不够 4 项，必合法；
- **P 长度 ⩾ 3，最后三个位置是 (z, u, t)**：共 g[z][u][t] 种，接上后新出现的窗口只有 (z, u, t, i) 这一个，合法 ⟺ a[z] ⊕ a[u] ⊕ a[t] ⊕ a[i] ≠ s。

$$g[u][t][i] = 1 + \Sigma\, g[z][u][t] \quad （对所有 z < u 且窗口异或 ≠ s 求和）$$

状态有 $O(n^3)$ 格，每格枚举 z 平均再花约 n/4，总复杂度 $O(n^4)$——n = 60 时实际约 5×10^5 次，实测 0.001 秒。

partial2_dp4.cpp · 三维 dp + 枚举第四项，$O(n^4)$

```
// T3 第二档：按"最后三个位置"记 dp，O(n^4)
// 适用：Subtask #1、#2，n <= 60，实际约 5*10^5 次操作
#include <cstdio>
const int MAXN = 62;              // 按本档上限 n <= 60 开；三维内存随 n^3 涨，别直接放大
const int MOD = 998244353;
int n, m, s;
int a[MAXN];
long long g[MAXN][MAXN][MAXN];    // 只用 u < t < i 的部分

int main() {
    freopen("count.in", "r", stdin);
```

```
    freopen("count.out", "w", stdout);
    scanf("%d %d %d", &n, &m, &s);
    for (int i = 1; i <= n; i++) scanf("%d", &a[i]);

    // 长度 1、2 的子序列必合法：n + C(n,2)
    long long ans = (long long)n * (n - 1) / 2 + n;
    for (int u = 1; u <= n; u++)
        for (int t = u + 1; t <= n; t++)
            for (int i = t + 1; i <= n; i++) {
                long long val = 1;                    // 前缀 {u,t}
                for (int z = 1; z < u; z++)           // 枚举倒数第 4 个位置 z
                    if ((a[z] ^ a[u] ^ a[t] ^ a[i]) != s)
                        val += g[z][u][t];
                g[u][t][i] = val % MOD;
                ans += g[u][t][i];
            }
    printf("%lld\n", ans % MOD);
    return 0;
}
```

不够 别被 $O(n^4)$ 吓到——把数组放大到 205，$n = 200$ 实测只要 0.05 秒，这一档其实还能再拿 25 分。它真正的天花板是三**维数组的内存**：$n = 600$ 时光被用到的部分就有约 810 MB、时间近 3 秒，双双超限；$n = 4000$ 时 6.4×10^{10} 格状态，想都别想。另外，**每格都重新枚举一遍 z** 也很浪费。下一档用第 0 节的钥匙：先查表省掉枚举，再顺手把第三维压掉。

4 第三档：压掉一维，查表定撞车

60 分

适用 subtask #1 ~ #3（合计 60 分）：$n \leqslant 200$。

观察一：撞车的 z 至多一个

上一档每格都枚举 z 逐个检查，其实根本不用查：窗口 (z, u, t, i) 里后三项已知，由第 0 节的钥匙，**会撞车的 z 值唯一确定**；又因为值互不相同，可以开一张“值 → 位置”的表 pos 直接查出它的位置：

$$z^{*} = \mathrm{pos}[\, s \oplus a[u] \oplus a[t] \oplus a[i] \,] \quad \text{（值不存在，或 } z^{*} \geqslant u\text{，都视为“不会撞车”）}$$

于是“逐个检查”变成“先全加起来，再减掉那一个”：$g[u][t][i] = 1 + \Sigma_{z<u}\, g[z][u][t] - g[z^{*}][u][t]$（$z^{*}$ 有效才减）。

观察二：u 只是用来检查的，可以压掉

答案只关心 $\Sigma_u\, g[u][t][i]$，既然检查可以查表完成，u 就不必留在状态里了。定义：

定义 $f[t][i]$（$t < i$）= 最后两个被选位置恰好是 t、i 的合法子序列个数（长度 $\geqslant 2$），也就是 $f[t][i] = 1 + \Sigma_{u<t}\, g[u][t][i]$（那个 1 是长度 2 的 {t, i}）。 答案 = n + 所有 f[t][i] 之和。

把 g 的转移式代进 f 的定义（下面所有 Σ 都对 $u < t$ 求和）：

$$f[t][i] = 1 + \Sigma_u\, g[u][t][i]$$

$$= 1 + \Sigma_u\, [\, 1 + \Sigma_{z<u}\, g[z][u][t] - g[z^*][u][t]\,] \quad \text{（代入 g 的转移式）}$$

$$= 1 + \Sigma_u\, f[u][t] - \Sigma_u\, g[z^*][u][t] \quad \text{（中括号前两项合起来正是 f[u][t]）}$$

最后那项里混着一个 g，用下面的引理把它也变成 f：

> **引理**　当 z^* 有效时，$g[z^*][u][t] = f[z^*][u]$。
> **证明**　按 g 的转移式，$g[z^*][u][t] = 1 + \Sigma\, g[w][z^*][u]$（对 $w < z^*$ 求和，唯一要排除的是满足 $a[w] = s \oplus a[z^*] \oplus a[u] \oplus a[t]$ 的 w）。把 z^* 的定义代进去（相同的项异或消掉）：$s \oplus (s \oplus a[u] \oplus a[t] \oplus a[i]) \oplus a[u] \oplus a[t] = a[i]$——唯一要排除的 w 恰好是**位置 i 自己**！i 在 t 的右边，根本不可能出现在前缀里，所以一项都不用排：$g[z^*][u][t] = 1 + \Sigma_{w<z^*}\, g[w][z^*][u]$，右边正是 $f[z^*][u]$ 的定义。∎

引理代入，得到只含 f 的转移式（记 $col[t] = \Sigma_{u<t}\, f[u][t]$，叫“列和”，边算边维护）：

$$f[t][i] = 1 + col[t] - \Sigma_{u<t}\, f[\,z^*\,][\,u\,] \quad (z^* = pos[s \oplus a[u] \oplus a[t] \oplus a[i]]，\text{有效才减})$$

配对 (t, i) 有 $O(n^2)$ 对，每对枚举 u 花 O(n)，总复杂度 $O(n^3)$，内存只要 $O(n^2)$——n = 200 时实际约 1.3×10^6 次，实测 0.004 秒。

手算样例的关键一格

以样例 a = (2,3,4,7,6)，s = 3 为例，看 f[3][5]（最后两项是位置 3、5，即值 4、6）：基础值 1 + col[3] = 1 + f[1][3] + f[2][3] = 1 + 1 + 2 = 4，对应前缀 {4}、{2,4}（这就是 f[1][3]）、{3,4} 和 {2,3,4}（这两个是 f[2][3]）。逐个查 u：

u	$z^* = pos[3 \oplus a[u] \oplus 4 \oplus 6]$	有效?	减去
1（值 2）	$pos[3\oplus2\oplus4\oplus6] = pos[3] = 2$	$z^* = 2 \geqslant u = 1$，无效	0
2（值 3）	$pos[3\oplus3\oplus4\oplus6] = pos[2] = 1$	$z^* = 1 < 2$，有效	f[1][2] = 1

f[3][5] = 4 − 1 = 3。被减掉的那个前缀正是 {2,3,4}——接上 6 就是样例里唯一不合法的 (2,3,4,6)。**dp 精确地把坏子序列挡在了门外**。用同样的方法可以把 10 格全部算完：

f[t][i]	i = 2	i = 3	i = 4	i = 5	行和
t = 1	1	1	1	1	4
t = 2		2	2	2	6
t = 3			4	3	7
t = 4				8	8

答案 = n + 行和之和 = 5 + (4 + 6 + 7 + 8) = **30** √。建议亲手验算其中一两格（比如 f[3][4]、f[4][5]），方法完全一样。

partial3_dp3.cpp · 二维 dp + 枚举修正项，$O(n^3)$

```cpp
// T3 第三档：压成二维 dp，修正项直接枚举，O(n^3)
// 适用：Subtask #1 ~ #3，n <= 200，约 1.3*10^6 次操作
#include <cstdio>
const int MAXN = 205;            // 按本档上限 n <= 200 开
const int MOD = 998244353;
int n, m, s;
int a[MAXN];
int pos[1 << 12];            // pos[v] = 值 v 在哪个位置（值互不相同）
long long f[MAXN][MAXN];     // f[t][i]：最后两个位置恰好是 t、i 的合法子序列数
long long col[MAXN];         // col[j] = sum_{u<j} f[u][j]，列和

int main() {
    freopen("count.in", "r", stdin);
    freopen("count.out", "w", stdout);
    scanf("%d %d %d", &n, &m, &s);
    for (int i = 1; i <= n; i++) {
        scanf("%d", &a[i]);
        pos[a[i]] = i;
    }
    long long ans = n % MOD;             // 长度为 1 的子序列
    for (int t = 1; t <= n; t++) {
        for (int i = t + 1; i <= n; i++) {
            long long val = (1 + col[t]) % MOD;
            for (int u = 1; u < t; u++) { // 枚举倒数第 2 个位置 u
                int z = pos[s ^ a[u] ^ a[t] ^ a[i]]; // 会撞车的倒数第 3 个值
                if (1 <= z && z < u)     // 它真的在 u 左边存在
                    val = (val - f[z][u] + MOD) % MOD;
            }
            f[t][i] = val;
            col[i] = (col[i] + val) % MOD;
            ans = (ans + val) % MOD;
        }
    }
    printf("%lld\n", ans);
    return 0;
}
```

不够　把数组放大到 1005，n = 1000 实测约 0.5 秒——这一档还能再拿 subtask #4 的 25 分，但已经贴着时限、几无余量；n = 4000 时约 1.1×10^{10} 次，要跑半分钟，彻底没戏。可公式里每一步都是 O(1) 查表，唯一的浪费是：**同一批修正，每对 (t, i) 都重新枚举一遍 u**。下一节把这层枚举消掉。

5　正解：修正项整批预处理

100 分

适用全部 subtask（100 分）：$n \leqslant 4000$，$m \leqslant 12$。

把修正项改写成“配对和”

盯住修正项的形式：$\Sigma_{u<t}\, f[z^*][u]$，其中 z^* 满足 $a[z^*] \oplus a[u] = s \oplus a[t] \oplus a[i]$。t 固定时，它只跟一个值 $w = s \oplus a[t] \oplus a[i]$ 有关。于是定义：

$$G_t[w] = \Sigma\, f[z][u]，\text{对所有 } z < u < t \text{ 且 } a[z] \oplus a[u] = w$$

转移立刻变成 $\mathbf{f[t][i] = 1 + col[t] - G_t[\, s \oplus a[t] \oplus a[i]\,]}$，每对 (t,i) 只查一次表，O(1)。

G 不用每行重算：只补“新解锁”的配对

从 G_{t-1} 到 G_t，多出来的只是“倒数第二项位置 u = t−1”的那些配对 (z, t−1)。对每个 w，配对里 z 的值必须满足 a[z] = w ⊕ a[t−1]，查一次 pos 表就知道 z 是谁：

$$G[w] \mathrel{+}= f[\, pos[w \oplus a[t-1]]\,][\, t-1\,] \quad （z 存在且 z < t−1 才加）$$

值域只有 $2^m \leqslant 4096$，所以每算完一行 f，花 $O(2^m)$ 就能把 G 维护好。每个配对 (z,u) 只在 u 被扫过时进 G 一次，不重不漏。（对照代码时注意：程序里是算完第 t 行之后才把 (z, t) 补进 G——对下一行来说，那就是这里的 G_{t+1}。）

复杂度

部分	量
填 f 表：$O(n^2/2)$，每格 O(1)	约 8×10^6
维护 G：每行 $O(2^m)$，共 n 行	约 1.6×10^7
合计	约 2.4×10^7，1 秒绰绰有余（实测约 0.1 秒）
内存	f 表 4000×4000×4B ≈ 64 MB（mod 意义下 int 足够）

full_solution.cpp · 修正项整批预处理，$O(n^2 + n \cdot 2^m)$

```cpp
// T3 正解：修正项用 G 表整批预处理，O(n^2 + n * 2^m)
// 适用：全部数据，n <= 4000，m <= 12；内存约 64 MB
#include <cstdio>
const int MAXN = 4005;
const int MOD = 998244353;
int n, m, s;
int a[MAXN];
int pos[1 << 12];               // pos[v] = 值 v 的位置，不存在则为 0
int f[MAXN][MAXN];              // f[t][i]（只用 t < i 的上三角），64 MB
int col[MAXN];                  // col[j] = sum_{u<j} f[u][j]
int G[1 << 12];                 // 上文的 G_t，边扫边维护

int main() {
    freopen("count.in", "r", stdin);
    freopen("count.out", "w", stdout);
    scanf("%d %d %d", &n, &m, &s);
    for (int i = 1; i <= n; i++) {
        scanf("%d", &a[i]);
        pos[a[i]] = i;
    }
    long long ans = n % MOD;              // 长度为 1 的子序列
    for (int t = 1; t <= n; t++) {
        int ds = col[t];                  // sum_{u<t} f[u][t]
        int base = s ^ a[t];
        for (int i = t + 1; i <= n; i++) {
            int val = 1 + ds - G[base ^ a[i]];
            if (val < 0) val += MOD;      // 可能为负，也可能恰好等于 MOD
            if (val >= MOD) val -= MOD;
```

```
            f[t][i] = val;
            col[i] += val;
            if (col[i] >= MOD) col[i] -= MOD;
            ans += val;
        }
        // 把"倒数第 2 个位置 = t"的配对补进 G，供后面的行使用
        int at = a[t];
        for (int w = 0; w < (1 << m); w++) {
            int u = pos[w ^ at];
            if (1 <= u && u < t) {
                G[w] += f[u][t];
                if (G[w] >= MOD) G[w] -= MOD;
            }
        }
    }
    printf("%lld\n", ans % MOD);
    return 0;
}
```

要点 递推顺序不能反：**先算完第 t 行的所有 f[t][i]，再把第 t 列补进 G**。反过来的话，(u, t) 这对会被提前算进 G，等于把“倒数第二项就是 t 自己”的配对也减掉了，必然多减。

6 易错点清单

1. **子序列不是子串**：挑中的元素原位置不用挨着；禁用窗口是“挑出后”新序列里的连续 4 项。
2. **长度 < 4 的子序列全部合法**：三维 dp 里答案先加 n + C(n,2)；二维 dp 里先加 n，转移里的那个 1 是“前缀只有 a[t]”。两处都别漏。
3. **撞车位置 z 的两个判断都不能少**：值要在序列里存在（pos 非 0），且必须 z < u——光存在不够，它还可能在 u 右边，那就轮不到它当倒数第三项。
4. **异或优先级**：C++ 里 `^` 的优先级比 `==` 低，判断必须写成 `((x ^ y ^ z ^ w) == s)`，漏括号就错了。
5. **减法取模**：`1 + ds - G[...]` 可能是负数，也可能恰好等于 MOD，两个方向都要补一次。
6. **表的大小按值域开**：G 和 pos 的大小是 2^m（最大 4096），不是 m，也不是 n。
7. **三维 dp 的内存随 n^3 涨**：n = 200 时 g 表约 69 MB 还行，n = 600 时被用到的部分就有约 810 MB，直接超限；正解的 f 表 4000×4000×4B ≈ 64 MB，mod 意义下开 int 即可。
8. **先算行，再补 G**：第 t 行的 f[t][i] 全部算完后，才能把第 t 列补进 G，顺序反了会多减。

7 总结

总结 数合法子序列，先从“记住结尾三个位置”的三维 dp 入手；再用钥匙性质“撞车值唯一 + pos 表查位置”把状态压成二维：f[t][i] = 1 + 列和 − 修正，其中修正恰好是 $f[z^*][u]$（第二层修正为空是关键引理）；最后把修正按 w = s ⊕ a[t] ⊕ a[i] 整批收进 G 表，每行 $O(2^m)$ 增量维护，得到 $O(n^2 + n\cdot 2^m)$ 的正解。 四档的递进：指数枚举太慢 → 三维 dp 会数了，但内存随 n^3 爆炸、每格重复枚举 → 查表定撞车 + 压掉一维，$O(n^3)$，但每对仍重复枚举修正 → 配对和整批预处理，消掉最后一重循环。